



Overview

Year / Semester	IV Year / 7 th Semester		Academic Year	2025-26
Laboratory Title	PARALLEL COMPUTING LAB		Laboratory Code	BCS702
Total Contact Hours	24 Hours		Duration of SEE	3 Hours
IA Marks	50 Marks		SEE Marks	50 Marks
Lab Manual Author	Prof. Sunita N. K	Sign -	Date	
Checked By	Prof. G.Y. Kamble	Sign -	Date	

Objectives

This course will enable to:

1. Explore the need for parallel programming
2. Explain how to parallelize on MIMD systems
3. To demonstrate how to apply MPI library and parallelize the suitable programs
4. To demonstrate how to apply OpenMP pragma and directives to parallelize the suitable programs
5. To demonstrate how to design CUDA program

Description

1.0 Learning Objectives

This course will enable to:

1. Explore the need for parallel programming
2. Explain how to parallelize on MIMD systems
3. To demonstrate how to apply MPI library and parallelize the suitable programs
4. To demonstrate how to apply OpenMP pragma and directives to parallelize the suitable programs
5. To demonstrate how to design CUDA program

2.0 Learning Outcomes

The student, after successful completion of the course, will be able to

CO	Course Outcome	RBT Level	POs
	Explain the need for parallel programming	L3	1,2,3,4,6,8,12
	Demonstrate parallelism in MIMD system.	L3	1,2,3,4,6,8,12
	Apply MPI library to parallelize the code to solve the given problem.	L3	1,2,3,4,6,8,12
	Apply OpenMP pragma and directives to parallelize the code to solve the given problem	L3	1,2,3,4,6,8,12
	Design a CUDA program for the given problem.	L3	1,2,3,4,6,8,12
Total Hours of instruction		100	



Prerequisites

Fundamentals of C Programming Language

Base Course

C Programming Language.

Introduction

INTRODUCTION OF PARALLEL COMPUTING

Parallel computing is a type of computation in which many calculations or processes are carried out simultaneously. It divides a large problem into smaller sub problems, which are then solved concurrently on multiple processors or computers. The main goal is to speed up processing and improve efficiency.

Definition

Parallel computing is the simultaneous use of multiple compute resources such as CPUs, GPUs, or cores to solve a computational problem. Instead of executing one task after another (as in serial computing), parallel computing performs many tasks at the same time.

Components:

1. **Tasks:** Independent or semi-independent units of work.
2. **Processors:** Computing units that execute tasks.
3. **Communication:** Data sharing among processors.
4. **Synchronization:** Coordination of tasks to ensure correct results.

Types of Parallelism

1. **Data Parallelism:** The same operation is performed on different chunks of data simultaneously.
2. **Task Parallelism:** Different tasks or operations are executed concurrently.
3. **Pipeline Parallelism:** Tasks are divided into stages, where different stages execute in parallel.

Advantages

- Faster execution time.
- Better resource utilization.
- Ability to handle large and complex problems.

Resources Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

General Instructions

1. Student should be punctual to the Lab
2. Required to prepare the Lab report every week
3. Required to maintain the Lab record properly
4. Should use the resources properly Student should be punctual to the Lab



5. Required to prepare the Lab report every week
6. Required to maintain the Lab record properly
7. Should use the resources properly

CONTENTS

Expt. No.	Experiments	Date Planned	Date Conducted
1	Write a OpenMP program to sort an array on n elements using both sequential and parallel mergesort(using Section). Record the difference in execution time.		
2	Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread. For example, if there are two threads and four iterations, the output might be the following: a. Thread 0 : Iterations 0 — 1 b. Thread 1 : Iterations 2 — 3		
3	Write a OpenMP program to calculate n Fibonacci numbers using tasks.		
4	Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times.		
5	Write a MPI Program to demonstration of MPI_Send and MPI_Recv.		
6	Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence.		
7	Write a MPI Program to demonstration of Broadcast operation.		
8	Write a MPI Program demonstration of MPI_Scatter and MPI_Gather		
9	Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD)		

Evaluation Scheme

Assessment Details (both CIE and SEE):

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

CIE for the practical component of the IPCC

- 15 marks for the conduction of the experiment and preparation of laboratory record, and 10 marks for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous



evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments' write-ups are added and scaled down to 15 marks.

- The laboratory test (duration 02/03 hours) after completion of all the experiments shall be conducted for 50 marks and scaled down to 10 marks.
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for 25 marks.
- The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

Reference

Text Books:

1. Peter S Pacheco, Matthew Malensek – An Introduction to Parallel Programming, second edition, Morgan Kauffman.
2. Michael J Quinn – Parallel Programming in C with MPI and OpenMp, McGrawHill.

Reference Books:

1. Calvin Lin, Lawrence Snyder – Principles of Parallel Programming, Pearson.
2. Barbara Chapman – Using OpenMP: Portable Shared Memory Parallel Programming, Scientific and Engineering Computation .
3. William Gropp, Ewing Lusk – Using MPI: Portable Parallel Programming, Third edition, Scientific and Engineering Computation



Experiments

1.0 Experiment (Sand Preparation)

Experiment No	Date Planed	Date Conducted	Marks
01.			

1.1 Learning Objectives

TITLE: Write an OpenMP program to sort an array on n elements using both sequential and parallel merge sort (using Section). Record the difference in execution time.

1.2 Material / Equipment Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

1.3 Theory / Hypothesis

OpenMP Program for Merge Sort (Sequential and Parallel)

Sorting is a fundamental operation in computer science used to arrange data in a specific order, usually ascending or descending. Among various sorting algorithms, Merge Sort is one of the most efficient algorithms based on the divide-and-conquer technique. It works by dividing an array into smaller subarrays, sorting them individually, and then merging them back into a single sorted array. In the sequential version, the merge sort algorithm recursively divides the array into two halves until single elements remain, then merges them in sorted order. This method is reliable but time-consuming for large datasets since it executes one recursive call at a time.

To enhance performance, OpenMP (Open Multi-Processing) can be used to parallelize the merge sort algorithm. OpenMP is a shared-memory parallel programming model that allows programmers to create multi-threaded applications using compiler directives (`#pragma omp`). In the parallel merge sort, multiple recursive calls are executed simultaneously by assigning them to different threads. The `#pragma omp parallel sections` directive divides the sorting task into sections, where each section runs independently on a separate thread.

Parallel execution allows the CPU cores to work concurrently, significantly reducing sorting time compared to the sequential approach. The efficiency gain depends on the number of threads and the size of the dataset.

This experiment demonstrates the power of parallelism by comparing the execution time of sequential and parallel merge sort. It helps students understand how multi-threading can optimize recursive algorithms and improve performance in shared-memory systems. Thus, OpenMP-based merge sort effectively showcases the concept of task parallelism, thread creation, synchronization, and performance analysis in real-world data processing applications.



1.4 Procedure / Program / Activity

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void merge(int arr[], int l, int m, int r) {
    int i = l, j = m + 1, k = 0;
    int temp[r - l + 1];

    while (i <= m && j <= r) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while (i <= m) temp[k++] = arr[i++];
    while (j <= r) temp[k++] = arr[j++];

    for (i = l, k = 0; i <= r; i++, k++)
        arr[i] = temp[k];
}

void sequentialMergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        sequentialMergeSort(arr, l, m);
        sequentialMergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void parallelMergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;

        #pragma omp parallel sections
        {
            #pragma omp section
            parallelMergeSort(arr, l, m);

            #pragma omp section
            parallelMergeSort(arr, m + 1, r);
        }

        merge(arr, l, m, r);
    }
}
```



```
int main() {  
    int n = 100000;  
    int arr1[n], arr2[n];  
  
    for (int i = 0; i < n; i++) {  
        arr1[i] = rand() % 1000;  
        arr2[i] = arr1[i];  
    }  
  
    double start, end;  
  
    start = omp_get_wtime();  
    sequentialMergeSort(arr1, 0, n - 1);  
    end = omp_get_wtime();  
    printf("Sequential Merge Sort Time: %f seconds\n", end - start);  
  
    start = omp_get_wtime();  
    parallelMergeSort(arr2, 0, n - 1);  
    end = omp_get_wtime();  
    printf("Parallel Merge Sort Time: %f seconds\n", end - start);  
  
    return 0;  
}
```

1.5 Results & Analysis

Output:

```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg1.c  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gcc -fopenmp prg1.c -o prg1  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ export OMP_NUM_THREADS=4 && ./prg1  
Sequential Merge Sort Time: 0.012787 seconds  
Parallel Merge Sort Time: 0.035695 seconds  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$
```

1.6 Remarks

FACULTY SIGNATURE



Experiments

1.0 Experiment (Sand Preparation)

Experiment No	Date Planed	Date Conducted	Marks
02			

1.1 Learning Objectives

TITLE: Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE =static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread. For example, if there are two threads and four iterations, the output might be the following:

- a. Thread 0 : Iterations 0 — 1
- b. Thread 1 : Iterations 2 — 3

1.2 Material / Equipment Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

1.3 Theory / Hypothesis

OpenMP Program to Divide Iterations Using Static Scheduling

In parallel programming, workload distribution among threads plays a crucial role in achieving better performance and efficiency. OpenMP (Open Multi-Processing) provides different scheduling techniques to control how loop iterations are assigned to multiple threads. The schedule clause in OpenMP specifies the strategy for distributing loop iterations. One commonly used scheduling type is static scheduling, where the iterations are divided into fixed-size chunks and assigned to threads before execution begins.

In this experiment, the directive `schedule(static,2)` is used, meaning that each thread executes two consecutive iterations of the loop at a time. The number of chunks is determined by dividing the total number of iterations by the chunk size. Static scheduling is suitable when all iterations take roughly the same amount of time because it minimizes runtime overhead and ensures predictable workload distribution.

The OpenMP directive `#pragma omp for schedule(static,2)` allows developers to clearly observe which thread executes which iterations. For example, in a loop with four iterations and two threads, Thread 0 executes iterations 0–1, while Thread 1 executes iterations 2–3. This approach demonstrates how OpenMP controls loop parallelism, thread assignment, and chunk handling. It helps understand how threads share the workload in shared-memory systems.



1.4 Procedure / Program / Activity

Program:

```
#include <stdio.h>
#include <omp.h>
int main() {
    int n,i;
    printf("Enter number of iterations: ");
    scanf("%d", &n);
    int thread_start[100], thread_end[100];
    for (i = 0; i < 100; i++) {
        thread_start[i] = -1;
        thread_end[i] = -1;
    }
    #pragma omp parallel for schedule(static,2)
    for (i = 0; i < n; i++) {
        int tid = omp_get_thread_num();

        if (thread_start[tid] == -1)
            thread_start[tid] = i;
        thread_end[tid] = i;
    }
    for (i = 0; i < 100; i++) {
        if (thread_start[i] != -1) {
            printf("Thread %d : Iterations %d -- %d\n", i, thread_start[i], thread_end[i]);
        }
    }
    return 0;
}
```

1.5 Results & Analysis

Output:

```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg2.c
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gcc -fopenmp prg2.c -o prg2
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ export OMP_NUM_THREADS=4 && ./prg2
Enter number of iterations: 6
Thread 0 : Iterations 0 -- 1
Thread 1 : Iterations 2 -- 3
Thread 2 : Iterations 4 -- 5
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$
```

1.6 Remarks

FACULTY SIGNATURE



Experiments

1.0 Experiment (Sand Preparation)

Experiment No	Date Planed	Date Conducted	Marks
03			

1.1 Learning Objectives

TITLE: Write an OpenMP program to calculate n Fibonacci numbers using tasks.

1.2 Material / Equipment Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

1.3 Theory / Hypothesis

OpenMP Program for Fibonacci Series Using Tasks

The Fibonacci sequence is a famous mathematical series where each number is the sum of the two preceding ones. The sequence starts with 0 and 1, and the next terms are obtained by repeatedly applying the formula

$$F(n) = F(n - 1) + F(n - 2).$$

Traditionally, Fibonacci numbers are calculated using a recursive function, which divides the problem into smaller sub problems. However, recursive computations involve repetitive function calls and overlapping sub problems, leading to increased execution time, especially for large values of n .

To overcome this limitation, OpenMP (Open Multi-Processing) can be used to parallelize the computation using the task directive. OpenMP tasks allow parts of a program to execute concurrently on multiple threads. Each recursive call can be represented as an independent task, enabling multiple Fibonacci computations to run simultaneously.

The `#pragma omp task` directive defines a new unit of work, while `#pragma omp taskwait` ensures synchronization between threads by waiting for all created tasks to complete. This task-based approach utilizes the concept of dynamic parallelism, where tasks are generated during program execution rather than being predefined.

By parallelizing Fibonacci computation, the workload is distributed among available processor cores, significantly reducing execution time compared to sequential recursion. This experiment demonstrates the concept of task creation, synchronization, and dependency in OpenMP. It also helps to understand how recursive problems can be efficiently solved using dynamic task generation.



1.4 Procedure / Program / Activity

Program:

```
#include <stdio.h>
#include <omp.h>
int fib(int n) {
    int x, y;
    if (n < 2)
        return n;
    #pragma omp task shared(x)
    x = fib(n - 1);
    #pragma omp task shared(y)
    y = fib(n - 2);
    #pragma omp taskwait
    return x + y;
}
int main() {
    int n;
    printf("Enter number of Fibonacci terms: ");
    scanf("%d", &n);
    printf("Fibonacci Series:\n");

    for (int i = 0; i < n; i++) {
        int result;
        #pragma omp parallel
        {
            #pragma omp single
            {
                result = fib(i);
            }
        }
        printf("%d ", result);
    }
    printf("\n");
    return 0;
}
```



1.5 Results & Analysis

Output:

```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg3.c
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gcc -fopenmp prg3.c -o prg3
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ export OMP_NUM_THREADS=4 && ./prg3
Enter number of Fibonacci terms: 6
Fibonacci Series:
0 1 1 2 3 5
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$
```

1.6 Remarks

FACULTY SIGNATURE



Experiments

1.0 Experiment (Sand Preparation)

Experiment No	Date Planed	Date Conducted	Marks
04			

1.1 Learning Objectives

TITLE: Write an OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times.

1.2 Material / Equipment Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

1.3 Theory / Hypothesis

OpenMP Program to Find Prime Numbers Using Parallel for Directive

Finding prime numbers is a classic problem in computer science and mathematics. A prime number is a number greater than 1 that has no positive divisors other than 1 and itself. The process of checking whether numbers within a given range are prime is computationally intensive, especially for large values of n .

In a sequential approach, the program checks each number one by one to determine if it is divisible by any smaller number. This process is simple but time-consuming because it runs in a single thread and does not utilize multiple CPU cores.

To improve performance, OpenMP (Open Multi-Processing) can be used to parallelize this operation. OpenMP is an API that supports shared-memory parallel programming by using compiler directives such as `#pragma omp parallel for`.

The parallel for directive allows loop iterations to be divided among multiple threads so that each thread independently checks a subset of numbers for primality. This results in a significant reduction in execution time as multiple numbers are processed simultaneously.

Each thread executes a portion of the loop without interfering with others, making the computation efficient and scalable. Proper synchronization or private variables are used to avoid data conflicts when threads access shared memory.

By recording both serial and parallel execution times, we can measure the performance improvement and understand the effect of multi-threading on CPU utilization. This experiment demonstrates how OpenMP can be applied to data-parallel problems and highlights the importance of load balancing, synchronization, and thread management in shared-memory parallel computing.



1.4 Procedure / Program / Activity

Program:

```
#include <stdio.h>
#include <math.h>
#include <omp.h>

int is_prime(int num) {
    if (num < 2) return 0;
    for (int i = 2; i <= sqrt(num); i++) {
        if (num % i == 0)
            return 0;
    }
    return 1;
}

int main() {
    int n;
    printf("Enter the value of n: ");
    scanf("%d", &n);

    printf("\nPrime numbers from 1 to %d:\n", n);
    for (int i = 1; i <= n; i++) {
        if (is_prime(i))
            printf("%d ", i);
    }
    printf("\n");

    double start_time, end_time;

    start_time = omp_get_wtime();
    for (int i = 1; i <= n; i++) {
        is_prime(i);
    }
    end_time = omp_get_wtime();
    printf("Serial Time: %f seconds\n", end_time - start_time);

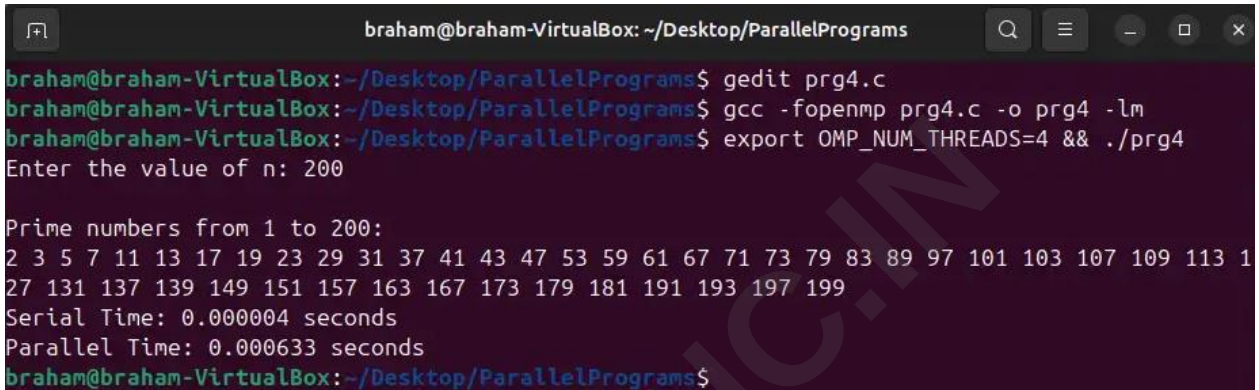
    start_time = omp_get_wtime();
    #pragma omp parallel for
    for (int i = 1; i <= n; i++) {
        is_prime(i);
    }
    end_time = omp_get_wtime();
```



```
printf("Parallel Time: %f seconds\n", end_time - start_time);  
  
return 0;  
}
```

1.5 Results & Analysis

Output:



```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg4.c  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gcc -fopenmp prg4.c -o prg4 -lm  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ export OMP_NUM_THREADS=4 && ./prg4  
Enter the value of n: 200  
  
Prime numbers from 1 to 200:  
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 1  
27 131 137 139 149 151 157 163 167 173 179 181 191 193 197 199  
Serial Time: 0.000004 seconds  
Parallel Time: 0.000633 seconds  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$
```

1.6 Remarks

FACULTY SIGNATURE



Experiments

1.0 Experiment (Sand Preparation)

Experiment No	Date Planed	Date Conducted	Marks
05			

1.1 Learning Objectives

TITLE: Write a MPI Program to demonstration of MPI_Send and MPI_Recv.

1.2 Material / Equipment Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

1.3 Theory / Hypothesis

MPI Program Demonstration of MPI_Send and MPI_Recv

In parallel and distributed computing, processes often need to exchange data to perform coordinated tasks. The Message Passing Interface (MPI) is a standardized communication protocol used to enable data exchange between multiple processes running on distributed systems. It provides a portable and efficient way to develop parallel applications that can run across multiple processors or nodes.

The fundamental concept in MPI is message passing, where processes communicate by explicitly sending and receiving messages. The two primary functions used for point-to-point communication are MPI_Send and MPI_Recv. MPI_Send is used by a process to send data to another process, while MPI_Recv is used to receive data from a specific process. Each message contains parameters such as data buffer, count, datatype, destination rank, tag, and communicator.

The sender and receiver must coordinate properly — the receiver must be ready to receive the message being sent; otherwise, deadlocks may occur. MPI ensures synchronization and correct message delivery through these communication routines.

This experiment demonstrates how one process (usually process 0) can send data to another process (say process 1) and how the receiving process retrieves and displays it. The communication can take place between any two processes identified by their rank within the communicator MPI_COMM_WORLD. By understanding and implementing MPI_Send and MPI_Recv, programmers can create more complex communication patterns like broadcast, scatter, and gather.

This experiment helps in understanding the concept of point-to-point communication, process synchronization, and data sharing in distributed-memory systems. It forms the foundation for building scalable and efficient parallel applications using MPI.



1.4 Procedure / Program / Activity

Program:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size;
    int number;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (size < 2) {
        if (rank == 0)
            printf("Please run with at least 2 processes.\n");
        MPI_Finalize();
        return 0;
    }
    if (rank == 0) {
        number = 100;
        printf("Process %d sending number %d to process 1\n", rank, number);
        MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else if (rank == 1) {
        MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received number %d from process 0\n", rank, number);
    }
    MPI_Finalize();
    return 0;
}
```



1.5 Results & Analysis

Output:

```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg5.c
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpicc prg5.c -o prg5
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpirun -np 2 ./prg5
Process 0 sending number 100 to process 1
Process 1 received number 100 from process 0
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$
```

1.6 Remarks

FACULTY SIGNATURE



Experiments

1.0 Experiment (Sand Preparation)

Experiment No	Date Planed	Date Conducted	Marks
06			

1.1 Learning Objectives

TITLE: Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence.

1.2 Material / Equipment Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

1.3 Theory / Hypothesis

MPI Program to Demonstrate Deadlock and Its Avoidance Using Point-to-Point Communication

In parallel programming, multiple processes often communicate with each other by sending and receiving messages. The Message Passing Interface (MPI) provides point-to-point communication functions such as MPI_Send and MPI_Recv for this purpose. However, improper ordering of these communication calls can lead to a condition called deadlock.

A deadlock occurs when two or more processes are waiting indefinitely for each other to send or receive a message. For example, if two processes both execute MPI_Send first without performing a corresponding MPI_Recv, each process will wait for the other to receive its message, causing the program to hang.

This experiment demonstrates how such a deadlock can occur in MPI programs when blocking communication calls are used in the wrong sequence. It helps in understanding the synchronization problems that arise in process communication.

Deadlock avoidance can be achieved by carefully altering the call sequence. One common method is to ensure that one process sends first while the other receives first. Alternatively, using non-blocking communication functions such as MPI_Isend and MPI_Irecv can also prevent deadlocks.

By modifying the order of MPI_Send and MPI_Recv calls, processes can complete their communication successfully without waiting indefinitely. This experiment provides a practical understanding of how message synchronization, ordering, and process coordination affect program correctness in distributed systems.

It also highlights the importance of designing safe communication patterns to ensure smooth and deadlock-free execution in parallel applications. Thus, MPI deadlock demonstration and avoidance form a key concept in building reliable and efficient parallel programs



1.4 Procedure / Program / Activity

Program:

```
#include <mpi.h>
#include <stdio.h>

// Change this to 1 for deadlock, 2 for deadlock avoidance
#define DEADLOCK_PART 2

int main(int argc, char *argv[]) {
    int rank, size, num = 123;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0)
            printf("Run with at least 2 processes.\n");
        MPI_Finalize();
        return 0;
    }

    #if DEADLOCK_PART == 1
        // ----- Part A: Deadlock -----
        if (rank == 0) {
            printf("Process 0 waiting to receive from Process 1...\n");
            MPI_Recv(&num, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            MPI_Send(&num, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        } else if (rank == 1) {
            printf("Process 1 waiting to receive from Process 0...\n");
            MPI_Recv(&num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            MPI_Send(&num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        }
    #endif

    #elif DEADLOCK_PART == 2
        // ----- Part B: Deadlock Avoidance -----
        if (rank == 0) {
            MPI_Send(&num, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
            MPI_Recv(&num, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            printf("Process 0 received back number: %d\n", num);
        } else if (rank == 1) {
            MPI_Recv(&num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            MPI_Send(&num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        }
    #endif
}
```



```
    printf("Process 1 received and sent back number: %d\n", num);  
}  
#endif  
MPI_Finalize();  
return 0;  
}
```

1.5 Results & Analysis

Output:

1.

```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg6.c  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpicc prg6.c -o prg6  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpirun -np 2 ./prg6  
Process 0 waiting to receive from Process 1...  
Process 1 waiting to receive from Process 0...
```

2.

```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg6.c  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpicc prg6.c -o prg6  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpirun -np 2 ./prg6  
Process 1 received and sent back number: 123  
Process 0 received back number: 123  
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$
```

1.6 Remarks

FACULTY SIGNATURE



Experiments

Experiment No	Date Planed	Date Conducted	Marks
07			

1.1 Learning Objectives

TITLE: Write a MPI Program to demonstration of Broadcast operation.

1.2 Material / Equipment Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

1.3 Theory / Hypothesis

Theory – MPI Program for Broadcast Operation

In parallel and distributed computing, efficient data sharing among processes is essential for coordinated computation. The Message Passing Interface (MPI) is a standardized library designed to enable communication among multiple processes running on different processors in a distributed-memory environment.

The broadcast operation in MPI is a collective communication function used when one process (called the *root*) needs to send the same data to all other processes within a communicator. The MPI function used for this purpose is `MPI_Bcast()`. In a broadcast, only the root process provides the data to be distributed, while all other processes receive a copy of that data. This eliminates the need for multiple point-to-point sends, thereby reducing communication overhead and improving efficiency. The syntax of the broadcast function is:

`MPI_Bcast(buffer, count, datatype, root, communicator);`

The broadcast operation ensures that all processes start with the same data, making it particularly useful in parallel algorithms where shared initial conditions or configuration parameters are required. For example, in numerical computations or matrix operations, a broadcast can be used to distribute input matrices or coefficients from one process to all others.

This experiment demonstrates how `MPI_Bcast` simplifies data distribution and ensures synchronization among processes. It also highlights the advantage of collective communication over multiple send/receive operations. By observing execution behavior, students understand the efficiency and scalability achieved through broadcast communication in distributed systems.



1.4 Procedure / Program / Activity

Program:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size;
    int number;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        number = 50;
        printf("Process %d broadcasting number %d to all other processes.\n", rank, number);
    }

    MPI_Bcast(&number, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received number %d\n", rank, number);

    MPI_Finalize();
    return 0;
}
```

1.5 Result & Analysis

Output:

```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg7.c
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpicc prg7.c -o prg7
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpirun -np 4 ./prg7
Process 0 broadcasting number 50 to all other processes.
Process 0 received number 50
Process 1 received number 50
Process 3 received number 50
Process 2 received number 50
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$
```

1.6 Remarks

FACULTY SIGNATURE



Experiments

Experiment No	Date Planed	Date Conducted	Marks
08			

1.1 Learning Objectives

TITLE: Write a MPI Program demonstration of MPI_Scatter and MPI_Gather.

1.2 Material / Equipment Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

1.3 Theory / Hypothesis

MPI Scatter and Gather Operations

MPI (Message Passing Interface) is a powerful standard used for communication among processes in a distributed-memory environment. It enables multiple processes running on different nodes to exchange data efficiently through message passing. In parallel computing, large datasets often need to be distributed among multiple processes for concurrent processing. This is achieved using collective communication functions such as MPI_Scatter and MPI_Gather.

The MPI_Scatter function is used to distribute chunks of data from one root process to all other processes in a communicator. Each process receives a unique portion of the original data array. This helps in dividing a large task into smaller independent parts that can be processed simultaneously. For example, if a dataset of 8 elements is scattered among 4 processes, each process receives 2 elements to work on. This parallel distribution of data enhances computational efficiency and minimizes processing time.

After each process performs its computation on the received data, the results need to be collected. This is done using the MPI_Gather function, which gathers data from all processes and assembles it back into a single array in the root process. Together, scatter and gather operations allow efficient data distribution and collection in distributed-memory systems. They form the foundation of many parallel applications such as matrix operations, simulations, and large-scale numerical computations.

1.4 Procedure / Program / Activity

Program:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size;
    int send_data[100], recv_data, gathered_data[100];
```



```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

if (rank == 0) {
    for (int i = 0; i < size; i++) {
        send_data[i] = i * 10;
    }
}

MPI_Scatter(send_data, 1, MPI_INT, &recv_data, 1, MPI_INT, 0, MPI_COMM_WORLD);
recv_data = recv_data + rank;
MPI_Gather(&recv_data, 1, MPI_INT, gathered_data, 1, MPI_INT, 0, MPI_COMM_WORLD);

if (rank == 0) {
    printf("Gathered data in root process:\n");
    for (int i = 0; i < size; i++) {
        printf("gathered_data[%d] = %d\n", i, gathered_data[i]);
    }
}

MPI_Finalize();
return 0;
}
```

1.5 Result & Analysis

Output:

```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg8.c
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpicc prg8.c -o prg8
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpirun -np 4 ./prg8
Gathered data in root process:
gathered_data[0] = 0
gathered_data[1] = 11
gathered_data[2] = 22
gathered_data[3] = 33
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$
```

1.6 Remarks

FACULTY SIGNATURE



Experiments

Experiment No	Date Planed	Date Conducted	Marks
09			

1.1 Learning Objectives

TITLE: Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_ MAX, MPI_MIN, MPI_SUM, MPI_PROD).

1.2 Material / Equipment Required

- The lab should have Intel based PC with Linux(OS)
- C Language under Linux / Windows environment.

1.3 Theory / Hypothesis

MPI Reduce and Allreduce Demonstration

In parallel computing, multiple processes often perform independent computations and need to combine their results to obtain a final outcome. The Message Passing Interface (MPI) provides efficient collective communication routines for such tasks. Among them, MPI_Reduce and MPI_Allreduce are two powerful operations used for data reduction across multiple processes.

MPI_Reduce performs a global reduction operation, such as sum, minimum, maximum, or product, on data distributed among all processes in a communicator. The result is then sent to a specified root process. This helps in collecting computed values from different processes into a single summarized result. On the other hand, MPI_Allreduce performs the same operation as MPI_Reduce but broadcasts the final reduced result to all processes, not just the root. This allows every process to have access to the global computed result.

The commonly used operations include MPI_SUM, MPI_PROD, MPI_MAX, and MPI_MIN. For example, summing distributed arrays, finding maximum values among processes, or calculating total products are easily handled by these functions. These collective operations minimize communication overhead and improve performance by avoiding multiple point-to-point message exchanges. They are highly scalable and optimized for distributed-memory architectures.

This experiment demonstrates how MPI_Reduce and MPI_Allreduce simplify the aggregation of results in parallel programs. By comparing both operations, students understand how data reduction and synchronization occur in MPI environments. Hence, these functions form an essential part of parallel algorithms used in scientific computing, simulations, and data analytics where multiple nodes work together to produce global results efficiently.



1.4 Procedure / Program / Activity

Program:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size;
    int value, sum, prod, max, min;
    int all_sum, all_prod, all_max, all_min;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    value = rank + 1;

    MPI_Reduce(&value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &prod, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &max, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &min, 1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD);

    MPI_Allreduce(&value, &all_sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_prod, 1, MPI_INT, MPI_PROD, MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_max, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_min, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("--- Results using MPI_Reduce (only root prints) ---\n");
        printf("Sum = %d\n", sum);
        printf("Product = %d\n", prod);
        printf("Max = %d\n", max);
        printf("Min = %d\n", min);
    }

    printf("Process %d has value %d\n", rank, value);
    printf("Process %d sees (MPI_Allreduce): Sum=%d, Prod=%d, Max=%d, Min=%d\n",
        rank, all_sum, all_prod, all_max, all_min);

    MPI_Finalize();
    return 0;
}
```



1.5 Result & Analysis

Output:

```
braham@braham-VirtualBox: ~/Desktop/ParallelPrograms
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ gedit prg9.c
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpicc prg9.c -o prg9
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$ mpirun -np 4 ./prg9
--- Results using MPI_Reduce (only root prints) ---
Sum = 10
Product = 24
Max = 4
Min = 1
Process 0 has value 1
Process 0 sees (MPI_Allreduce): Sum=10, Prod=24, Max=4, Min=1
Process 1 has value 2
Process 2 has value 3
Process 2 sees (MPI_Allreduce): Sum=10, Prod=24, Max=4, Min=1
Process 3 has value 4
Process 3 sees (MPI_Allreduce): Sum=10, Prod=24, Max=4, Min=1
Process 1 sees (MPI_Allreduce): Sum=10, Prod=24, Max=4, Min=1
braham@braham-VirtualBox:~/Desktop/ParallelPrograms$
```

1.6 Remarks

FACULTY SIGNATURE